

Table des matières

1	Introduction	5
1.1	Présentation de l'organisme d'accueil	5
1.2	Problématique et objectifs	6
1.3	Analyse et spécification des besoins	6
1.3.1	Identification des acteurs	6
1.3.2	Spécification des besoins fonctionnels (Use Case global) .	7
1.3.3	Spécification des besoins non fonctionnels	8
1.4	Méthodologie adoptée	8
1.4.1	Méthodologie Scrum	8
1.4.2	Principes de Scrum	9
1.5	Langage de modélisation UML	10
1.6	Conclusion	12
2	Introduction	14
2.1	Étude et critique de l'existant	14
2.1.1	Processus actuel de gestion des candidatures	14
2.1.2	Analyse du processus par diagramme BPMN	14
2.2	Critique de l'existant	15
2.2.1	Avantages	15
2.2.2	Inconvénients	15
2.3	Problématique	15
2.4	Étude de solutions équivalentes (Onboarding)	16
2.4.1	Introduction	16
2.5	Présentation des solutions existantes	16
2.5.1	Solution 1 : BambooHR	16
2.5.2	Solution 2 : Workday	16
2.5.3	Solution 3 : Zoho Recruit	16
2.5.4	Solution 4 : Odoo	17
2.5.5	Tableau comparatif des solutions	17
2.5.6	Analyse comparative	17
2.6	Conclusion	18
3	Introduction	20
3.1	Architecture globale	20
3.1.1	Architecture logique	20
3.1.2	Architecture physique	22
3.2	Diagrammes de cas d'utilisation raffinés	23
3.2.1	Cas d'utilisation de l'Administrateur	23
3.2.2	Description des fonctionnalités	23
3.3	Cas d'utilisation du Recruteur	24
3.3.1	Description des fonctionnalités	24
3.4	Cas d'utilisation du Candidat	25
3.4.1	Description des fonctionnalités	25
3.5	Conception détaillée	26

3.5.1	Diagramme de classes	26
3.5.2	Diagrammes de séquence	27
4	Conclusion	32
5	Introduction	34
6	Environnement de développement	34
6.1	Environnements de développement intégrés (IDE)	34
6.2	Frameworks et technologies	35
6.3	Base de données	36
6.4	Outils de test et de collaboration	36
6.5	Outil de modélisation UML	38
6.6	Interfaces de l'application	39

Introduction

Depuis l’essor du réseau numérique mondial, les plateformes en ligne ont largement dépassé leur rôle initial de simple consultation. Elles offrent aujourd’hui des fonctionnalités interactives permettant d’adapter les réponses aux attentes des utilisateurs. Cette transformation a poussé les organisations à intégrer des solutions numériques afin d’optimiser leurs performances et d’améliorer les échanges avec leur clientèle, que ce soit en présentiel ou à distance.

Dans ce contexte, notre projet consiste à concevoir et développer une solution informatique dédiée à l’administration du capital humain au sein de l’entreprise Empreinte Pub, qui rencontre occasionnellement des difficultés liées à la lenteur de ses processus administratifs. Parmi les fonctions essentielles liées à cette administration figurent notamment le recrutement, la rémunération, l’évaluation ainsi que la planification des emplois et des compétences [1].

Ce travail met particulièrement l’accent sur le traitement des dossiers de candidature. En effet, l’entreprise est confrontée chaque année à un nombre élevé de demandes, nécessitant la mise en place d’un outil permettant d’automatiser et de rationaliser leur gestion afin d’améliorer l’efficacité globale.

Ce rapport présente les différentes étapes du projet ainsi que les résultats obtenus. Il est structuré en plusieurs chapitres : le premier expose le contexte général, la problématique et l’analyse des besoins ; le second propose une étude théorique accompagnée d’une analyse critique de l’existant et d’une comparaison des solutions similaires ; le troisième détaille la conception du système à travers son architecture et ses modèles UML ; enfin, le quatrième décrit la phase de réalisation en mettant en avant l’environnement technique ainsi que les interfaces développées. Une conclusion générale viendra clôturer ce travail.

Chapitre 1 : Contexte Général du Projet

1 Introduction

Dans ce chapitre, nous situons le projet dans son environnement global en présentant les éléments essentiels à sa compréhension. Nous débutons par une description de l'organisme d'accueil. Par la suite, une analyse de l'existant est menée afin de mettre en évidence les limites actuelles et de préciser les objectifs que la solution proposée devra satisfaire.

1.1 Présentation de l'organisme d'accueil

Empreinte Pub est une entreprise spécialisée dans les activités d'impression et de communication publicitaire. Elle propose des services variés couvrant la conception graphique, la production de supports imprimés tels que les affiches, brochures et flyers, ainsi que le développement de solutions de communication visuelle adaptées aux besoins spécifiques de ses clients.

Forte de son expertise dans ce domaine, l'entreprise accompagne les organisations dans la mise en valeur de leur image de marque, en s'appuyant sur des supports à la fois créatifs et de haute qualité.

Dans le cadre de ce projet, nous avons intégré cette structure avec pour objectif de contribuer à la modernisation de ses processus internes. Plus précisément, notre travail a porté sur le développement d'une application dédiée au recrutement en ligne. Cette solution vise à automatiser le traitement des candidatures, depuis leur soumission jusqu'à leur gestion, tout en améliorant la fluidité, l'efficacité et la traçabilité du processus de recrutement.



FIGURE 1 – Logo Empreinte Pub

1.2 Problématique et objectifs

Bien que le processus de recrutement existant permette d'assurer une certaine fiabilité dans la gestion des candidatures, il présente plusieurs limites en termes d'efficacité. En effet, les outils utilisés pour répondre aux besoins en ressources humaines sont souvent dispersés entre différents services, ce qui entraîne un manque de centralisation et une faible intégration avec le système d'information. Cette fragmentation, combinée à une forte dépendance aux traitements manuels, engendre des lenteurs et réduit la performance globale du processus.

Par ailleurs, les méthodes de recrutement traditionnelles, telles que le recours aux agences locales ou aux médias classiques, ne garantissent pas toujours une adéquation optimale entre les profils des candidats et les besoins des entreprises. De plus, ces approches limitent la visibilité des offres d'emploi, empêchant ainsi certains candidats qualifiés d'accéder aux opportunités disponibles.

Dans ce contexte, il devient essentiel de mettre en place une solution plus performante, capable d'accélérer et d'optimiser le processus de recrutement. C'est dans cette optique que s'inscrit l'adoption du recrutement en ligne, qui constitue une alternative moderne, efficace et mieux adaptée aux exigences actuelles.

1.3 Analyse et spécification des besoins

1.3.1 Identification des acteurs

Dans le cadre de la modélisation UML (Unified Modeling Language), un acteur désigne une entité externe — utilisateur ou système — interagissant avec l'application. Pour notre système de gestion de recrutement, plusieurs acteurs principaux ont été identifiés :

- **Administrateur (Ressources Humaines)**

Cet acteur assure la supervision globale du système. Il est chargé de la gestion des comptes utilisateurs, du paramétrage de l'application ainsi que de la publication et du suivi des offres d'emploi. Il dispose également d'outils d'analyse lui permettant de générer des statistiques relatives aux performances du système et au traitement des candidatures. Son rôle est essentiel pour garantir le bon fonctionnement et la cohérence du processus de recrutement.

- **Candidats**

Les candidats représentent les utilisateurs finaux du système. Ils utilisent la plateforme pour consulter les offres d'emploi et soumettre leur candidature en fournissant les informations nécessaires, telles que le CV et la lettre de motivation. Le système leur permet également de suivre l'évolution de leur dossier tout au long du processus de recrutement et de participer aux différentes étapes de sélection, notamment les entretiens et les éventuels tests.

— **Recruteur (Responsable)**

Le recruteur intervient principalement dans la gestion opérationnelle du recrutement. Il est responsable de la diffusion des offres d'emploi, de l'identification des profils correspondant aux besoins de l'entreprise et de la sélection des candidats. Il procède au tri des candidatures selon des critères définis, puis organise les entretiens et assure le suivi des échanges avec les candidats retenus.

1.3.2 Spécification des besoins fonctionnels (Use Case global)

Les besoins fonctionnels décrivent les services que le système doit fournir ainsi que les interactions entre les différents acteurs.

Vue globale du système Le diagramme de cas d'utilisation global présente une vue d'ensemble des principales fonctionnalités du système ainsi que les interactions entre les acteurs (Administrateur, Recruteur et Candidat).

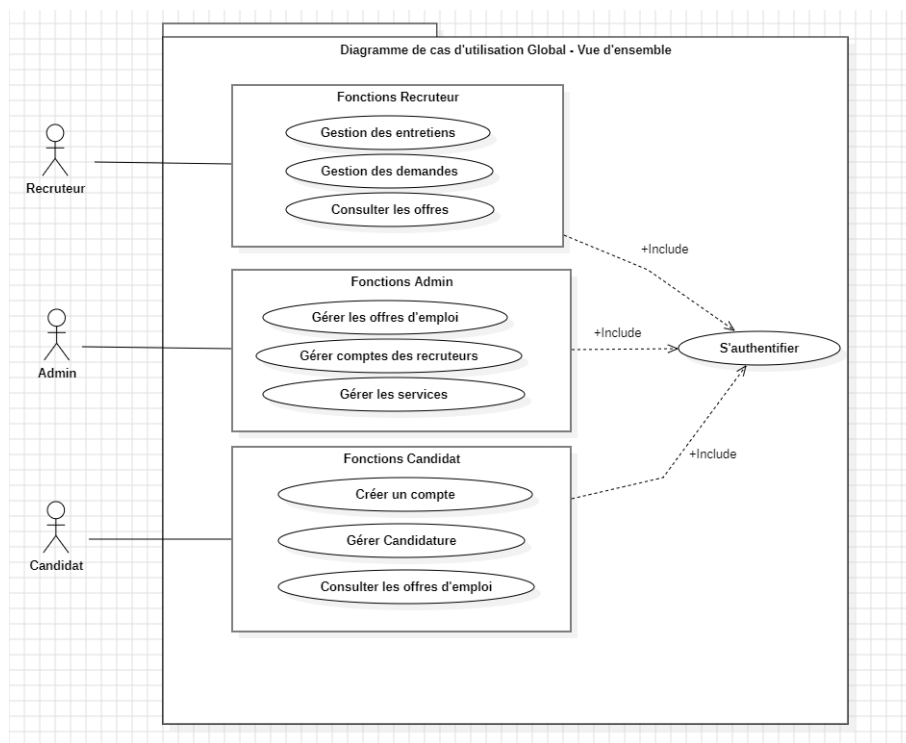


FIGURE 2 – Diagramme de cas d'utilisation global

Description générale Le système permet :

- La gestion des offres d'emploi

- La gestion des candidatures
- La planification des entretiens
- Le suivi du processus de recrutement

1.3.3 Spécification des besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes techniques et qualitatives auxquelles le système doit répondre afin de garantir sa performance, sa sécurité et sa maintenabilité.

Ergonomie et utilisabilité Le système doit offrir une interface conviviale :

- Interface simple, intuitive et facile à utiliser.
- Navigation fluide entre les différentes fonctionnalités.

Sécurité Le système doit garantir la protection des données et des accès :

- Authentification sécurisée des utilisateurs (login/mot de passe).
- Gestion des rôles (Administrateur, Recruteur, Candidat).
- Protection des données sensibles (CV, informations personnelles).

Compatibilité Le système doit être accessible sur différents environnements :

- Compatible avec les navigateurs web modernes.
- Adapté aux différents types d'écrans (responsive design).

Performance et rapidité Le système doit garantir des temps de réponse optimaux :

- Chargement rapide des pages et des données.
- Traitement efficace des requêtes utilisateurs.

Disponibilité Le système doit être accessible en permanence :

- Disponibilité continue de l'application (24h/24 et 7j/7).
- Réduction des interruptions et des temps d'indisponibilité.

1.4 Méthodologie adoptée

La réussite d'un projet informatique repose sur l'adoption d'une démarche méthodologique rigoureuse tout au long de son cycle de vie. Le choix de cette méthodologie doit être effectué en tenant compte des spécificités du projet, notamment la charge de travail, les ressources humaines mobilisées et les contraintes de délai, afin d'assurer une organisation optimale et une exécution efficace.

1.4.1 Méthodologie Scrum

Scrum est un cadre de travail agile largement utilisé dans le développement des projets informatiques. Il se distingue par sa flexibilité et sa capacité à s'adapter aux changements tout au long du cycle de développement.

Le terme Scrum, issu du rugby et signifiant « mêlée », fait référence à une organisation collaborative où l'équipe avance de manière coordonnée vers un objectif commun. Cette analogie illustre la dynamique d'adaptation continue du projet et l'amélioration progressive du produit.

Ainsi, Scrum repose sur une approche empirique basée sur l'expérience, la transparence et l'amélioration continue.

1.4.2 Principes de Scrum

Le fonctionnement de Scrum repose sur plusieurs éléments clés organisés de manière itérative [2] :

User stories Les user stories représentent les besoins fonctionnels exprimés du point de vue utilisateur. Elles décrivent les fonctionnalités attendues avec un titre, une description et une estimation.

Product backlog Le product backlog est une liste priorisée regroupant l'ensemble des besoins fonctionnels du projet sous forme de user stories.

Sprint Un sprint est une itération de développement d'une durée généralement comprise entre deux et quatre semaines.

Sprint planning Cette réunion permet de sélectionner les user stories à réaliser durant le sprint.

Sprint backlog Le sprint backlog correspond à l'ensemble des tâches que l'équipe s'engage à réaliser durant un sprint.

Daily meeting Il s'agit d'une réunion quotidienne courte permettant de suivre l'avancement et d'identifier les blocages.

Burndown chart Cet outil graphique permet de suivre l'évolution du travail restant au cours du sprint.

Sprint review À la fin du sprint, les fonctionnalités développées sont présentées afin de valider leur conformité.



FIGURE 3 – Déroulement de la méthodologie Scrum

1.5 Langage de modélisation UML

UML est un langage de modélisation graphique standard largement utilisé en génie logiciel pour analyser, concevoir, représenter et documenter des systèmes informatiques [3]. Il permet de décrire de manière visuelle les différents éléments d'un système ainsi que leurs relations, sans se limiter à une technologie d'implémentation spécifique.

Ce langage met à disposition un ensemble de diagrammes permettant de modéliser un système selon plusieurs perspectives. D'une part, les diagrammes structurels représentent l'organisation statique du système, notamment à travers les classes, les composants ou encore les objets. D'autre part, les diagrammes comportementaux décrivent le fonctionnement dynamique du système, en mettant en évidence les interactions entre les acteurs et l'application, comme les cas d'utilisation, les séquences ou les activités.

Dans le cadre de ce projet, UML est exploité pour :

- analyser et formaliser les besoins fonctionnels à l'aide des diagrammes de cas d'utilisation
- modéliser l'architecture logique du système via les diagrammes de classes
- représenter les échanges et interactions entre les utilisateurs et le système
- améliorer la communication et la compréhension entre les différentes parties prenantes du projet.

Ainsi, UML constitue un outil fondamental dans la démarche de conception, garantissant une meilleure structuration du système et une vision claire de son

fonctionnement global.

1.6 Conclusion

Ce chapitre a permis de présenter le contexte général du projet ainsi que les différentes problématiques liées au processus de recrutement. Il a également permis d'identifier les besoins du système, les acteurs principaux et les objectifs de la solution proposée. De plus, la méthodologie Scrum et le langage UML ont été introduits afin d'assurer une organisation et une conception structurées du projet.

Le chapitre suivant sera consacré à l'étude théorique du projet. Il portera sur l'analyse du processus actuel de recrutement, l'identification de ses limites ainsi que l'étude comparative des solutions existantes afin de mieux positionner la solution proposée.

Chapitre 2 : Étude Théorique

2 Introduction

Dans ce chapitre, nous nous intéressons à l'étude théorique du projet à travers une analyse du processus actuel de gestion des candidatures au sein de l'entreprise. Cette étude permet d'identifier les limites et les difficultés rencontrées dans le système existant.

Par ailleurs, une étude comparative de plusieurs solutions de recrutement et d'onboarding disponibles sur le marché sera réalisée afin d'analyser leurs fonctionnalités, leurs avantages ainsi que leurs limites. Cette démarche permettra de mieux justifier les choix adoptés pour la conception de la solution proposée.

2.1 Étude et critique de l'existant

2.1.1 Processus actuel de gestion des candidatures

Dans l'entreprise Empreinte Pub, le traitement des candidatures repose sur une organisation hybride combinant des supports physiques et numériques. Les informations liées aux candidats sont centralisées de manière manuelle, ce qui implique une manipulation successive des dossiers avant leur exploitation.

Le traitement des candidatures s'appuie principalement sur des outils bureautiques tels que Microsoft Excel, utilisé comme moyen de suivi et de première analyse des profils reçus. Cette approche permet au responsable des ressources humaines d'avoir une vision globale des candidatures, mais reste dépendante d'un traitement manuel important.

Sur la base de cette analyse, une sélection progressive des profils est effectuée en fonction de leur adéquation avec les besoins exprimés. Les candidats retenus sont ensuite contactés afin de planifier des entretiens d'évaluation, généralement organisés par voie électronique.

Ce mode de fonctionnement, bien qu'opérationnel, met en évidence une forte dépendance aux interventions humaines ainsi qu'une absence de centralisation structurée des données, ce qui limite l'efficacité globale du processus.

2.1.2 Analyse du processus par diagramme BPMN

Afin de mieux comprendre le fonctionnement du processus actuel, une modélisation sous forme de diagramme d'activité BPMN (Business Process Model and Notation) a été réalisée [4]. Cette représentation permet de visualiser les différentes étapes du processus ainsi que les interactions entre les acteurs impliqués dans la gestion des candidatures.

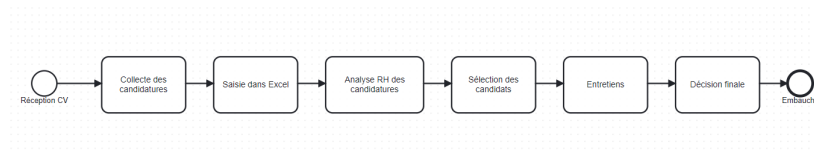


FIGURE 4 – Diagramme BPMN du processus actuel de recrutement

L'analyse du diagramme met en évidence que le processus débute par la réception des candidatures provenant de différentes sources. Celles-ci sont ensuite centralisées manuellement dans un fichier de suivi.

Une phase de tri est ensuite effectuée afin d'identifier les profils correspondant aux besoins du poste. Les candidats présélectionnés passent ensuite par une phase d'entretien, qui constitue l'étape principale d'évaluation.

Enfin, une décision de recrutement est prise et le candidat retenu est informé de son intégration dans l'entreprise.

2.2 Critique de l'existant

L'analyse du système actuel permet d'identifier plusieurs avantages ainsi que des limites importantes.

2.2.1 Avantages

- Simplicité de mise en œuvre du processus de recrutement.
- Utilisation d'outils standards (Excel, email) ne nécessitant pas de formation complexe.
- Flexibilité dans le traitement manuel des candidatures.

2.2.2 Inconvénients

- Absence de centralisation des données, entraînant une gestion fragmentée des candidatures.
- Forte dépendance aux traitements manuels, source d'erreurs et de perte d'informations.
- Temps de traitement élevé du fait de la manipulation manuelle des dossiers.
- Difficulté de suivi et de traçabilité des candidatures.
- Manque d'automatisation du processus de sélection.

2.3 Problématique

Le processus actuel de recrutement, bien qu'opérationnel, présente des limites en termes d'efficacité et de centralisation. L'utilisation d'outils dispersés ainsi que le traitement manuel des candidatures rendent le processus lent et peu optimisé.

Dans ce contexte, il devient nécessaire d'envisager une solution informatisée permettant de centraliser les candidatures, d'automatiser certaines étapes du processus et d'améliorer la traçabilité ainsi que la rapidité de traitement.

2.4 Étude de solutions équivalentes (Onboarding)

2.4.1 Introduction

Afin de mieux positionner notre solution et d'identifier les pratiques existantes sur le marché, une étude comparative de plusieurs outils de gestion des ressources humaines a été réalisée. Ces solutions intègrent des fonctionnalités liées au recrutement ainsi qu'au processus d'onboarding, permettant d'accompagner l'intégration des nouveaux employés.

2.5 Présentation des solutions existantes

2.5.1 Solution 1 : BambooHR

BambooHR est une solution SaaS dédiée à la gestion des ressources humaines. Elle propose des fonctionnalités de gestion des candidatures, de suivi des employés et d'onboarding. Elle est particulièrement adaptée aux petites et moyennes entreprises.



FIGURE 5 – Logo de BambooHR

2.5.2 Solution 2 : Workday

Workday est une plateforme complète de gestion des ressources humaines destinée aux grandes entreprises. Elle offre des fonctionnalités avancées de recrutement, d'analyse des données RH et d'intégration des employés.



FIGURE 6 – Logo de Workday

2.5.3 Solution 3 : Zoho Recruit

Zoho Recruit est un outil spécialisé dans le recrutement. Il permet de gérer les candidatures, automatiser les processus de sélection et améliorer le suivi des candidats.



FIGURE 7 – Logo de Zoho Recruit

2.5.4 Solution 4 : Odoo

Odoo est une solution open source proposant plusieurs modules, dont la gestion des ressources humaines. Il offre des fonctionnalités de recrutement et d'onboarding avec un haut niveau de personnalisation.



FIGURE 8 – Logo de Odoo

2.5.5 Tableau comparatif des solutions

Critère	BambooHR	Workday	Zoho Recruit	Odoo
Gestion des candidatures	✓	✓	✓	✓
Onboarding	✓	✓	✗	✓
Automatisation	Moyenne	Élevée	Élevée	Moyenne
Facilité d'utilisation	Bonne	Complexe	Bonne	Moyenne
Coût	Payant	Payant	Freemium	Gratuit
Personnalisation	Moyenne	Faible	Bonne	Très élevée

TABLE 1 – Comparaison des solutions existantes

2.5.6 Analyse comparative

L'analyse des différentes solutions met en évidence que les outils existants offrent des fonctionnalités avancées pour la gestion du recrutement et de l'onboarding. Cependant, ces solutions présentent certaines limites. Les plateformes

commerciales telles que Workday ou BambooHR sont souvent coûteuses et peu accessibles pour les petites structures.

D'autre part, certaines solutions comme Zoho Recruit se concentrent principalement sur le recrutement sans couvrir l'ensemble du cycle d'intégration des employés. Les solutions open source telles que Odoo offrent une flexibilité importante, mais nécessitent des compétences techniques pour leur configuration et leur maintenance.

2.6 Conclusion

Cette étude montre qu'aucune solution existante ne répond parfaitement aux besoins spécifiques de l'entreprise. Les limitations identifiées, notamment en termes de coût, de complexité et de personnalisation, justifient la mise en place d'une solution adaptée, permettant d'optimiser le processus de recrutement et d'améliorer l'intégration des nouveaux employés.

Dans le chapitre suivant, nous aborderons la phase de conception du système. Cette partie sera consacrée à la modélisation de l'application ainsi qu'à la définition de son architecture et de ses différents composants.

Chapitre 3 : Conception

3 Introduction

Une fois l'analyse des besoins et l'étude des solutions existantes réalisées, il devient nécessaire de définir une conception claire et structurée du système proposé. Cette étape permet de transformer les besoins identifiés en une architecture cohérente, capable de garantir le bon fonctionnement de l'application ainsi que la bonne organisation de ses différents modules.

Ce chapitre est consacré à la conception de la plateforme. Nous commencerons par présenter l'architecture logique et physique afin d'exposer la répartition des composants du système, les technologies adoptées ainsi que les mécanismes de communication entre les différentes couches de l'application. Nous aborderons ensuite les diagrammes de cas d'utilisation raffinés pour chaque acteur dans le but de détailler les différentes fonctionnalités disponibles. Par la suite, nous présenterons le diagramme de classes qui décrit la structure des entités et les relations qui les relient. Enfin, les diagrammes de séquence permettront d'illustrer le comportement dynamique du système à travers les principaux scénarios d'exécution.

3.1 Architecture globale

La conception de l'application représente une phase déterminante du projet, car elle permet de définir la structure générale du système ainsi que l'organisation de ses différents composants. Cette étape vise principalement à établir une architecture cohérente répondant aux besoins fonctionnels et techniques identifiés durant l'analyse, tout en assurant la fiabilité, la sécurité et la facilité d'évolution de la solution développée.

Dans cette optique, l'application a été élaborée selon une architecture en couches, permettant de répartir les responsabilités entre plusieurs niveaux indépendants. Cette séparation améliore la maintenabilité du système, facilite les futures évolutions et optimise la gestion des différentes fonctionnalités de l'application.

3.1.1 Architecture logique

L'architecture logique illustre la manière dont les composants logiciels de l'application sont structurés et interagissent entre eux. Le système adopté repose sur une architecture à trois niveaux (3-Tiers), largement utilisée dans les applications web en raison de sa simplicité d'organisation et de son efficacité.

Cette architecture se compose des couches suivantes :

Couche de présentation Cette couche représente la partie visible de l'application destinée aux utilisateurs. Elle a été réalisée avec le framework Angular afin de fournir une interface moderne, interactive et responsive. Son rôle principal consiste à afficher les informations, gérer les actions effectuées par les utilisateurs et assurer la communication avec les services du serveur.

Exécutée côté client dans le navigateur web, cette couche contribue à offrir une meilleure expérience utilisateur grâce à une navigation rapide et une mise à jour dynamique des données.

Couche métier La couche métier constitue le noyau fonctionnel de l'application. Elle est développée avec Spring Boot et prend en charge l'ensemble des traitements liés aux règles de gestion. Cette couche reçoit les requêtes envoyées depuis l'interface utilisateur, traite les opérations nécessaires puis communique avec la base de données pour récupérer ou enregistrer les informations.

Elle assure également plusieurs fonctionnalités importantes telles que le contrôle des données, la gestion des erreurs ainsi que la sécurisation des accès. L'intégration de Spring Security permet notamment de gérer les mécanismes d'authentification et d'autorisation afin de protéger les ressources de l'application contre les accès non autorisés.

Couche de données Cette couche est dédiée à la gestion et à la conservation des données de l'application. Le système de gestion de base de données MySQL a été choisi pour assurer le stockage des informations de manière fiable et structurée.

La couche de données permet d'effectuer les différentes opérations de manipulation des données, telles que l'ajout, la modification, la suppression et la consultation des enregistrements. Elle garantit également la cohérence, l'intégrité et la sécurité des données utilisées par l'ensemble du système.

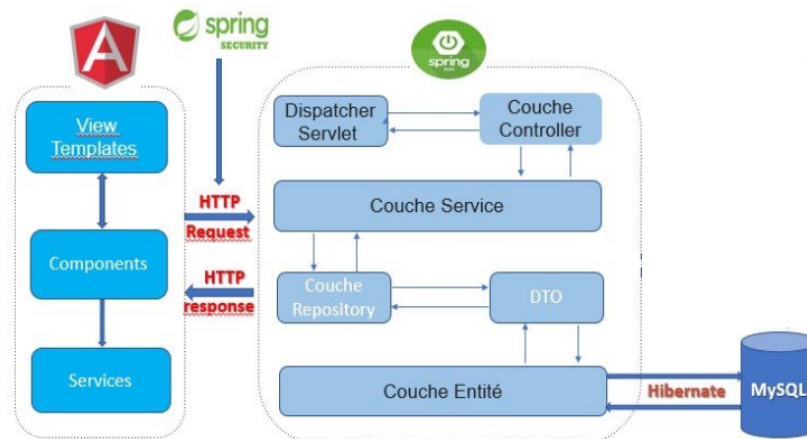


FIGURE 9 – Architecture logique

3.1.2 Architecture physique

L'architecture physique correspond à l'organisation technique des composants déployés au sein de l'application ainsi qu'aux mécanismes de communication utilisés entre eux. Le système développé adopte un modèle client-serveur permettant de séparer clairement la partie interface utilisateur de la partie traitement métier.

La partie front-end est assurée par Angular, exécuté au niveau du navigateur web. Cette couche se charge de l'affichage des pages, de la gestion des actions utilisateur et de l'envoi des requêtes vers le serveur. Quant à la partie back-end, elle est implémentée avec Spring Boot, dont le rôle consiste à traiter les demandes reçues, exécuter les opérations métier et interagir avec la base de données.

La communication entre le client et le serveur s'effectue via des services web basés sur le protocole HTTP. Les informations échangées sont transportées sous forme de données JSON, un format léger et largement utilisé pour assurer l'interopérabilité entre les applications web modernes.

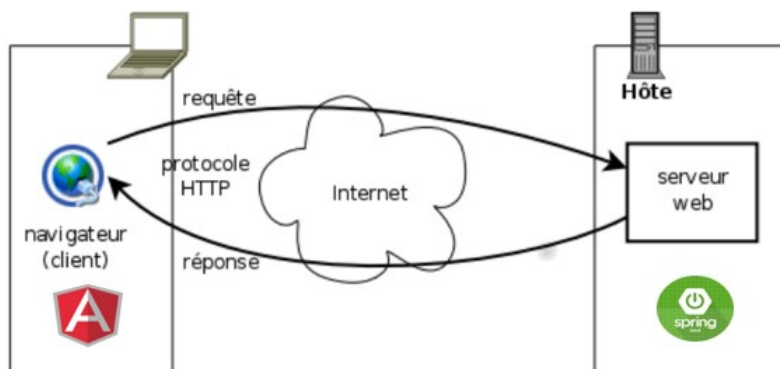


FIGURE 10 – Architecture logique

3.2 Diagrammes de cas d'utilisation raffinés

3.2.1 Cas d'utilisation de l'Administrateur

Le diagramme suivant présente les différentes fonctionnalités accessibles à l'administrateur au sein du système.

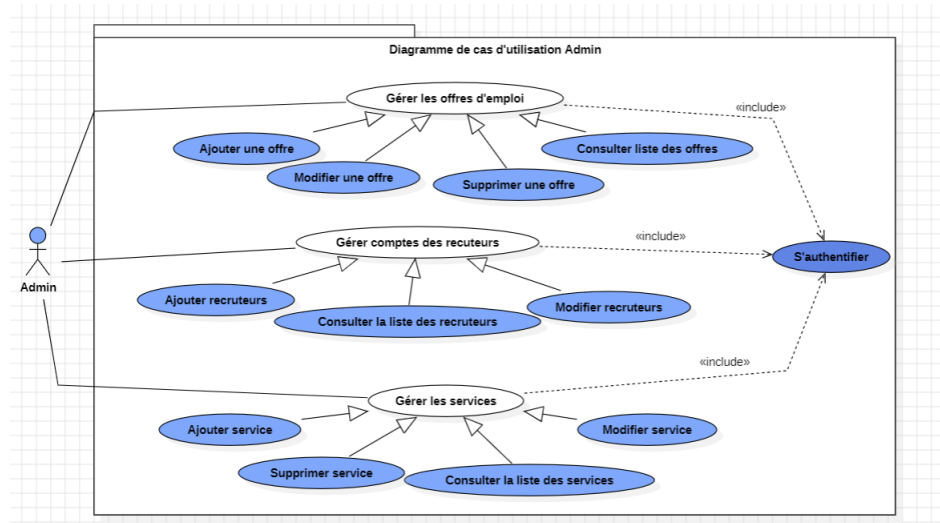


FIGURE 11 – Diagramme de cas d'utilisation de l'administrateur

3.2.2 Description des fonctionnalités

Gestion des secteurs

- Créer, modifier et supprimer un secteur.

Gestion des recruteurs

- Créer un compte recruteur.
- Affecter un recruteur à un secteur.
- Modifier et supprimer un compte recruteur.

Gestion des offres

- Créer et publier des offres d'emploi.
- Affecter un recruteur responsable.
- Archiver les offres expirées.

3.3 Cas d'utilisation du Recruteur

Le diagramme suivant illustre les principaux cas d'utilisation associés au recruteur.

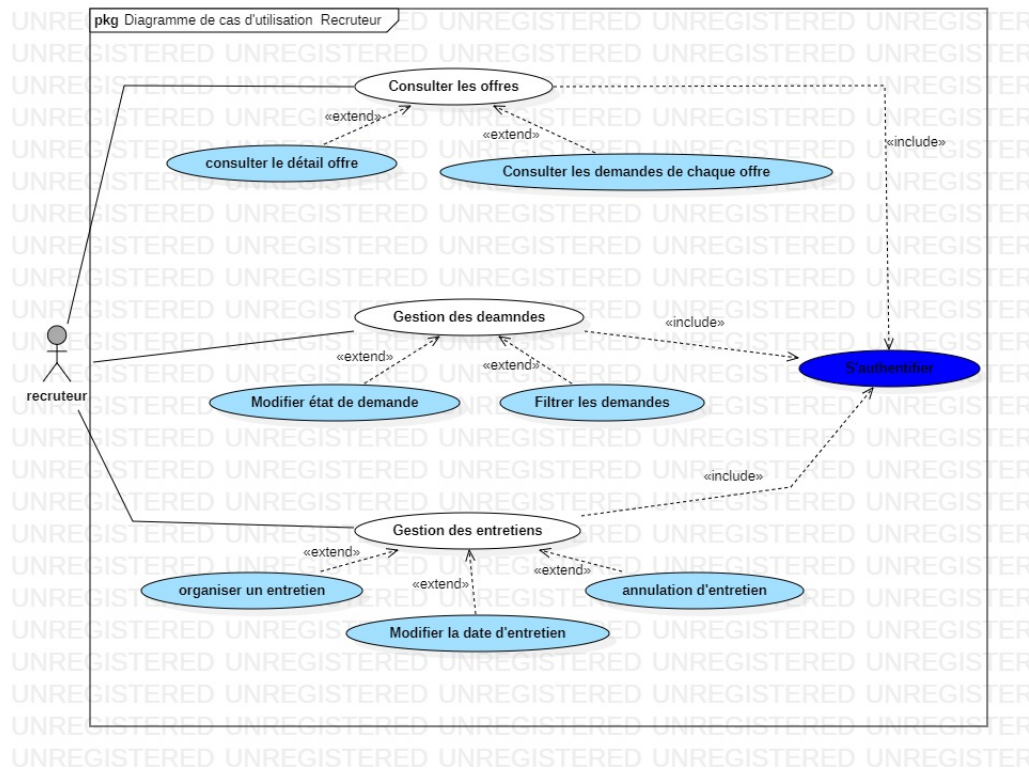


FIGURE 12 – Diagramme de cas d'utilisation du recruteur

3.3.1 Description des fonctionnalités

Consultation des offres

- Consulter les offres associées à son secteur.

Gestion des candidatures

- Consulter les candidatures.
- Évaluer et trier les candidats.
- Accepter ou rejeter une candidature.

Gestion des entretiens

- Planifier des entretiens.
- Envoyer des convocations.

3.4 Cas d'utilisation du Candidat

Le diagramme suivant décrit les fonctionnalités et interactions accessibles au candidat dans l'application.

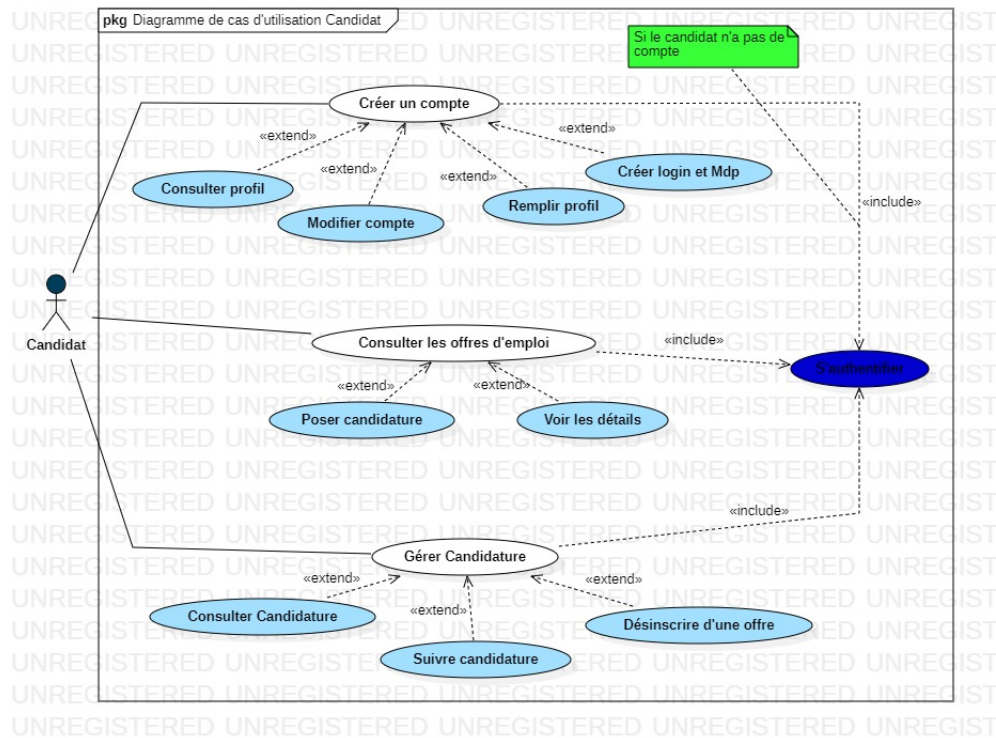


FIGURE 13 – Diagramme de cas d'utilisation du candidat

3.4.1 Description des fonctionnalités

Gestion du compte

- Créer et gérer son compte.

Consultation des offres

- Rechercher et consulter les offres.

Candidature

- Soumettre une candidature.
- Passer un test si nécessaire.

Suivi des candidatures

- Suivre l'état des candidatures.

3.5 Conception détaillée

Cette section est consacrée à la description détaillée de la conception du système. Afin de représenter les différents composants de l'application de manière claire et structurée, nous nous appuyons sur le langage de modélisation UML, qui constitue un outil de référence dans l'analyse et la conception des systèmes informatiques.

3.5.1 Diagramme de classes

Le diagramme de classes représente l'un des principaux diagrammes structuraux proposés par UML. Il permet de modéliser l'architecture statique de l'application en mettant en évidence les différentes classes qui composent le système, leurs attributs, leurs opérations ainsi que les relations établies entre elles, telles que les associations, les héritages ou les dépendances.

Ce diagramme joue un rôle important dans la phase de conception puisqu'il fournit une vision globale et cohérente de l'organisation interne du système. Il facilite également la compréhension des interactions entre les différents composants logiciels et constitue une base essentielle pour l'implémentation de l'application.

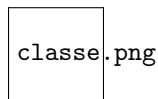


FIGURE 14 – Diagramme de classe

La figure 15 illustre le diagramme de classes réalisé dans le cadre de notre projet.

3.5.2 Diagrammes de séquence

Afin d'analyser le comportement fonctionnel de notre application, nous utilisons les diagrammes de séquence UML. Ces diagrammes permettent de représenter de manière chronologique les interactions réalisées entre les acteurs et les différents composants du système lors de l'exécution d'une fonctionnalité donnée.

Ils mettent principalement en évidence la circulation des messages échangés entre les objets ainsi que l'enchaînement des traitements effectués au sein de l'application. Cette modélisation dynamique facilite la compréhension du fonctionnement interne du système et illustre la collaboration entre les différentes couches logiques de l'architecture.

Dans cette partie, nous présentons les scénarios de séquence les plus importants liés aux fonctionnalités principales de notre application.

Processus d'authentification

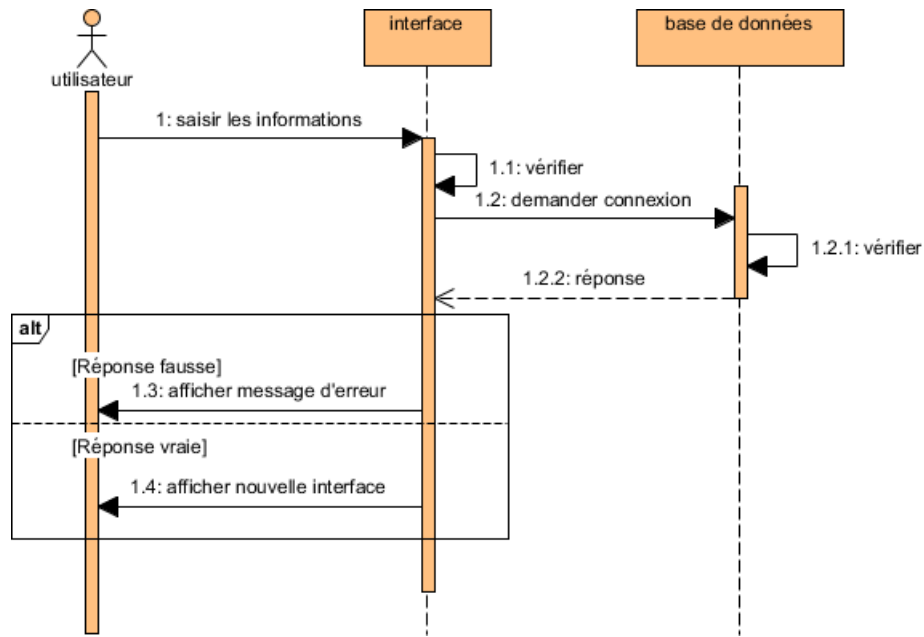


FIGURE 15 – Diagramme de séquence du mécanisme d'authentification

La figure 14 présente le diagramme de séquences modélisant le mécanisme d'authentification des utilisateurs au sein de l'application. Ce diagramme implique trois acteurs : l'utilisateur, l'interface et la base de données, et décrit le flux d'interactions depuis la saisie des identifiants jusqu'à l'obtention ou au refus de l'accès.

Acteurs et prérequis **Acteurs concernés :** les candidats, les recruteurs et l'administrateur système, tous désignés sous le terme générique *utilisateur* dans ce diagramme.

Finalité : garantir un accès sécurisé à l'application et aux fonctionnalités propres à chaque profil utilisateur.

Condition préalable : l'utilisateur doit disposer d'un compte préalablement enregistré et actif dans le système.

Déroulement nominal Le processus d'authentification se déroule selon les étapes suivantes :

1. L'utilisateur accède à l'interface de connexion et saisit ses informations d'identification (message 1 : *saisir les informations*).
2. L'interface procède à une première vérification de la cohérence des données saisies (message 1.1 : *vérifier*), puis transmet une demande de connexion à la base de données (message 1.2 : *demander connexion*).
3. La base de données vérifie l'existence et la validité du compte utilisateur (message 1.2.1 : *vérifier*), puis retourne une réponse à l'interface (message 1.2.2 : *réponse*).

Bloc alternatif (alt) : traitement de la réponse En fonction du résultat retourné par la base de données, deux scénarios sont envisagés dans le fragment combiné de type **alt** :

- **[Réponse fausse]** : si les identifiants fournis sont incorrects ou inexistants, l'interface affiche un message d'erreur à l'utilisateur (message 1.3 : *afficher message d'erreur*), l'invitant à vérifier ses informations et à renouveler sa tentative.
- **[Réponse vraie]** : si les informations d'identification sont validées avec succès, l'interface redirige l'utilisateur vers l'espace fonctionnel correspondant à son rôle (message 1.4 : *afficher nouvelle interface*), qu'il s'agisse de l'espace candidat, recruteur ou administrateur.

Résultat attendu À l'issue de ce processus, l'utilisateur authentifié se voit accorder l'accès aux fonctionnalités de l'application conformément à ses droits et privilèges. En cas d'échec répété, des mécanismes de sécurité complémentaires, tels que le verrouillage temporaire du compte ou la réinitialisation du mot de passe, peuvent être envisagés dans les extensions du système.

Gestion des offres

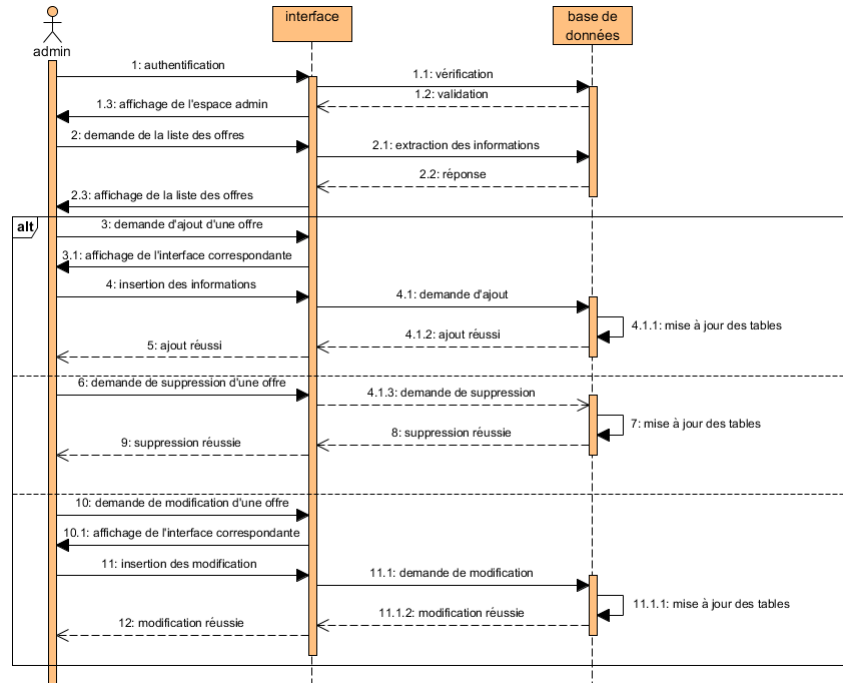


FIGURE 16 – Diagramme de séquences de la gestion des offres

La figure 15 présente le diagramme de séquences relatif à la gestion des offres d'emploi, impliquant trois acteurs principaux : l'administrateur (*admin*), l'interface utilisateur et la base de données. Ce diagramme modélise les interactions liées aux opérations d'authentification, de consultation, d'ajout, de suppression et de modification des offres.

Authentification Le processus débute par une étape d'authentification : l'administrateur soumet ses identifiants à l'interface (message 1), qui les transmet à la base de données pour vérification (message 1.1). Une fois la validation effectuée (message 1.2), l'espace d'administration est affiché à l'utilisateur (message 1.3).

Consultation de la liste des offres Après connexion, l'administrateur demande la liste des offres disponibles (message 2). L'interface interroge la base de données (message 2.1 : extraction des informations), qui retourne les données

correspondantes (message 2.2 : réponse). La liste est ensuite affichée à l'administrateur (message 2.3).

Bloc alternatif (alt) : opérations CRUD Les opérations suivantes sont regroupées dans un fragment combiné de type **alt** (alternative), ce qui signifie qu'une seule d'entre elles est exécutée selon le choix de l'administrateur.

- **Ajout d'une offre** : L'administrateur demande l'ajout d'une offre (message 3), ce qui provoque l'affichage du formulaire correspondant (message 3.1). Il saisit les informations requises (message 4), et l'interface transmet une demande d'insertion à la base de données (message 4.1). Cette dernière met à jour les tables (message 4.1.1) et confirme le succès de l'opération (message 4.1.2). Un message de confirmation est finalement renvoyé à l'administrateur (message 5 : ajout réussi).
- **Suppression d'une offre** : L'administrateur sélectionne une offre à supprimer (message 6). L'interface envoie une demande de suppression à la base de données (message 4.1.3), qui procède à la mise à jour des tables (message 7) et retourne une confirmation (message 8 : suppression réussie). L'interface notifie l'administrateur du succès de l'opération (message 9).
- **Modification d'une offre** : L'administrateur sollicite la modification d'une offre existante (message 10), ce qui déclenche l'affichage du formulaire d'édition prérempli (message 10.1). Après modification des champs, l'administrateur soumet les nouvelles données (message 11). L'interface transmet la demande de modification à la base de données (message 11.1), qui met à jour les tables (message 11.1.1) et confirme la réussite (message 11.1.2). Un message de succès est retourné à l'administrateur (message 12 : modification réussie).

Ce diagramme illustre ainsi une architecture trois-tiers classique où l'interface joue le rôle d'intermédiaire entre l'utilisateur et la persistance des données, garantissant la cohérence et l'intégrité des informations à chaque opération.

4 Conclusion

Ce chapitre a été consacré à l'étude de conception de l'application en mettant l'accent sur l'organisation globale du système et les différents modèles UML réalisés. Les diagrammes de cas d'utilisation ont permis de définir les interactions entre les utilisateurs et l'application, tandis que les diagrammes de classes et de séquence ont servi à représenter la structure interne du système ainsi que le déroulement des traitements principaux.

Le travail de conception effectué représente une étape importante avant la phase d'implémentation, puisqu'il permet de clarifier les besoins techniques et fonctionnels du projet tout en assurant une meilleure organisation du développement.

Dans le chapitre suivant, nous aborderons la phase de réalisation de la solution à travers la présentation de l'environnement de développement, des technologies choisies ainsi que des différentes interfaces de l'application développée.

Chapitre 4 : Réalisation

5 Introduction

Ce chapitre présente la phase de réalisation du projet ainsi que les choix techniques effectués durant le développement de l'application. Il met en évidence les différents outils, frameworks et technologies utilisés pour assurer la conception et l'implémentation de la solution proposée.

La première partie est dédiée à la présentation de l'environnement de développement et des technologies adoptées pour le développement du front-end et du back-end. La seconde partie expose les différentes interfaces de l'application afin de montrer les fonctionnalités implémentées et le déroulement des principales opérations offertes aux utilisateurs.

6 Environnement de développement

Cette section présente l'ensemble des outils, technologies et environnements utilisés dans le cadre du développement de ce projet. Le choix de ces outils a été guidé par des critères de performance, de maintenabilité et d'adéquation avec les exigences techniques de l'application. L'environnement de développement repose sur une combinaison d'outils de conception, d'édition de code, de gestion de versions, de tests et de frameworks modernes, formant ainsi une chaîne outillée cohérente et professionnelle.

6.1 Environnements de développement intégrés (IDE)

Visual Studio Code Visual Studio Code (VS Code) est un éditeur de code source léger et extensible, développé par Microsoft. Il offre une prise en charge native de nombreux langages de programmation, notamment TypeScript et JavaScript, utilisés dans le cadre du développement de la partie front-end avec Angular. Grâce à son écosystème riche d'extensions, il facilite l'autocomplétion, le débogage, la gestion du contrôle de version et l'intégration des outils de formatage de code.



FIGURE 17 – Logo de Visual Studio Code, éditeur de code utilisé pour le développement front-end

IntelliJ IDEA IntelliJ IDEA est un environnement de développement intégré (IDE) conçu par JetBrains, reconnu pour sa puissance dans le développement d'applications Java et Spring. Il a été utilisé dans ce projet pour le développement de la couche back-end reposant sur Spring Boot. IntelliJ IDEA propose des fonctionnalités avancées telles que la refactorisation intelligente, la navigation contextuelle dans le code, l'intégration native de Maven et Gradle, ainsi qu'un débogueur performant.

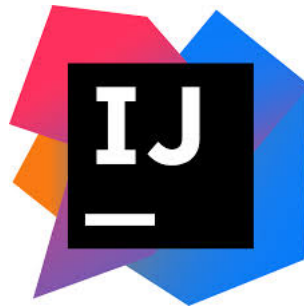


FIGURE 18 – Logo d’IntelliJ IDEA, IDE utilisé pour le développement back-end Spring Boot

6.2 Frameworks et technologies

Spring Boot Spring Boot est un framework Java open-source basé sur le framework Spring, permettant de créer des applications back-end robustes et autonomes avec une configuration minimale. Dans ce projet, il constitue le socle du serveur applicatif, en charge de l'exposition des API REST, de la gestion de la logique métier, de la sécurité et de la communication avec la base de données. Son architecture modulaire et son système d'injection de dépendances contribuent à la maintenabilité et à l'évolutivité de l'application.



FIGURE 19 – Logo de Spring Boot, framework back-end utilisé pour le développement des API REST

Angular Angular est un framework front-end open-source développé et maintenu par Google, basé sur le langage TypeScript. Il a été adopté dans ce projet pour la conception de l'interface utilisateur sous forme d'une application monopage (SPA — *Single Page Application*). Angular permet une organisation modulaire du code, une gestion efficace des composants, des services et du routage, ainsi qu'une liaison de données bidirectionnelle facilitant les interactions dynamiques entre l'interface et le serveur.



FIGURE 20 – Logo d'Angular, framework front-end utilisé pour le développement de l'interface utilisateur

6.3 Base de données

MySQL MySQL est un système de gestion de bases de données relationnelles (SGBDR) open-source, largement adopté dans les applications web. Il a été retenu dans ce projet pour assurer la persistance des données, notamment la gestion des utilisateurs, des offres d'emploi et des candidatures. Son intégration avec Spring Boot via Spring Data JPA et Hibernate permet une communication fluide et sécurisée entre la couche applicative et la couche de persistance.

SQL Le langage SQL (*Structured Query Language*) est le langage standard utilisé pour interagir avec la base de données relationnelle MySQL. Il a été employé pour la définition du schéma de la base de données (DDL), la manipulation des données (DML) ainsi que pour les requêtes de consultation et d'agrégation. La maîtrise de SQL a permis d'optimiser les performances des requêtes et d'assurer l'intégrité référentielle des données.

6.4 Outils de test et de collaboration

Postman Postman est une plateforme dédiée aux tests et à la documentation des API. Dans ce projet, il a été utilisé pour tester et valider les endpoints REST exposés par le back-end Spring Boot, en simulant des requêtes HTTP de



FIGURE 21 – Logo de MySQL, SGBDR utilisé pour la gestion et la persistance des données



FIGURE 22 – Représentation du langage SQL utilisé pour la gestion des requêtes et du schéma de base de données

type GET, POST, PUT et DELETE. Postman permet également d'organiser les requêtes en collections et d'automatiser les scénarios de test, garantissant ainsi la fiabilité et la conformité des API développées.



FIGURE 23 – Logo de Postman, outil de test et de validation des API REST

GitHub GitHub est une plateforme de gestion de versions et de collaboration basée sur le système de contrôle de version Git. Elle a été utilisée tout au long du projet pour héberger le dépôt du code source, gérer les branches de développement, suivre l'historique des modifications et coordonner les contributions. GitHub garantit la traçabilité du code et constitue un outil essentiel pour la gestion de projet en équipe.



FIGURE 24 – Logo de GitHub, plateforme de gestion de versions et de collaboration utilisée dans le projet

6.5 Outil de modélisation UML

StarUML StarUML est un outil de modélisation UML (*Unified Modeling Language*) professionnel, permettant la conception de diagrammes structurels et comportementaux. Il a été employé dans la phase d'analyse et de conception du projet pour produire l'ensemble des diagrammes UML, notamment les diagrammes de cas d'utilisation, de classes, de séquences et d'activités. Ces modèles ont servi de support à la conception de l'architecture logicielle avant le démarrage de l'implémentation.



FIGURE 25 – Logo de StarUML, outil de modélisation utilisé pour la conception des diagrammes UML du projet

6.6 Interfaces de l'application

Références

- [1] M. Bernard, *Gestion des Ressources Humaines et Digitalisation*, Dunod, 2021.
- [2] Ken Schwaber et Jeff Sutherland, *The Scrum Guide*, disponible en ligne : <https://scrumguides.org/>
- [3] Object Management Group (OMG), *Unified Modeling Language (UML)*, disponible en ligne : <https://www.uml.org/>
- [4] Object Management Group (OMG), *Business Process Model and Notation (BPMN)*, disponible en ligne : <https://www.omg.org/spec/BPMN/>